LLM Preliminaries

It means how we can improve LLM. There are 2 ways to improve.

1. **Improve the prompt**:
   We improve the output by giving the model better instructions.
   Example:
   - Poor prompt: Explain AI.
   - Better prompt: Explain AI in simple English in 3 bullet points with one real-life example.

   Here we can change the prompt, not the model internal properties.

2. **Tuning (Fine-tuning):**
   We train an already-trained model further on a specific dataset so it performs better for a particular task.
   Example: **YAMNET** (Yet another mobile network), it a pre-trained model, that we can train on **ESC-50** dataset (Environmental Sound Classification).

Tuning Types

1. **Supervised Tuning**: We train an already-trained model using examples of inputs and their correct, expected outputs.
   Example: We want an AI to answer customer-support questions.
   Training example
   Input: What is the refund period?
   Correct output: You can request a refund within 10 days of delivery.

Here train model with similar examples or here we tell model how good the answer looks like.

2. **Reinforcement Learning**: We further train a model using a reward, that tells us how good its generated solutions are.
   Example: We want an AI to solve mathematical problem correctly.
   Question: What is 25 x 4?
   - The model generates an answer: 100.
   - Evaluate the answer
   - Correct answer → positive reward,
   - Incorrect answer → lower reward or penalty

   During reward the model weight is adjusted to get more likely reward.
   During penalty the model weight is adjusted to get less likely reward.

Weight → It is a parameter (variable) that tells how importance the input is.

Example: Suppose we build an AI that decides whether to jump over an obstacle.

If the weight is '2' → The AI doesn't jump often enough and hits an obstacle.

Now the weight is adjusted to get less likely output, or weight is adjusted so that it won't hit the obstacle.

Gradient → How changing the weight will affect the error, whether it will reduce or increase the error.

Error = Predicted – Actual

Reinforcement Learning on Hardware

Training is divided into 2 Phases.

1. **Rollout Phase**: AI perform action on real hardware and observes what happens next.
   Example:
   - Robot arm tries to pick up a cup.
   - It misses the cup.
   - The system records the action and result.

   In Rollout Phase AI gains experience.
2. **Update Phase**: It uses the experience and rewards to update the AI model's weights.
   Example:
   - It analyzes why the robot missed the cup.
   - It adjusts the model to improve its future actions.

Why does Hardware matter?

When training on a real robot, actions take physical time, and mistakes can damage equipment.

vLLM (Virtual Large Language Model)

vLLM is a tool that runs Large Language Models (LLMs) efficiently so they can generate responses quickly and serve multiple users.

Example: Suppose you make a chat bot for students in college.

Now if there are few requests you can handle, but handling large requests can be difficult.

So vLLM manages the request and memory efficiently, allowing model to serve multiple requests. It removes extra memory waste. E.g. In web applications we have REST APIs, whenever we give server new request it does not remember the previous request.

Reinforcement Learning on Math

$$\mathbf{Q_{new} = Q_{old} + \alpha\, \Delta_o \log \pi_\Theta(a)}$$

Q_{new} = New weight.

Q_{old} = Old weight.

α = Learning weight (A number that decides how much we change the weight)

small → weight change slowly.

Large → weight change Fastly.

A = Advantage (How much better or worse the action was then expected).

$\pi_\Theta(a)$ = Probability that AI choose Action A.

$\log\pi_\Theta(a)$ = Log of that probability.

Δ_o = Tell how to adjust the weight.

Example: Imagine an AI decide whether to jump an obstacle or not.

PPO -- Proximal Policy Optimization

PPO is an RL technique that improves an AI's decisions while preventing its behaviour from changing too much in a single update.

Example: Imagine you're training a robot to pick up a cup.

Step 1: Try

The robot attempts to pick up the cup.

Step 2: Get feedback

It receives a reward for successfully picking up the cup.

Step 3: Improve carefully

PPO updates the model so successful actions become more likely.

Example: You ask an AI: 'What is 5 × 6?'

First attempt = 35

The AI gives the wrong answer and receives a low reward.

After learning = **30**

The AI learns to make the correct answer more likely.

PPO updates the model carefully so that its behaviour doesn't change too drastically in one update.

GRPO -- Group Relative Policy Optimization

GRPO improves an AI by generating multiple answers to the same question, comparing their rewards, and learning to favour the better answers.

Imagine you're training an AI to solve a maths question: What is 5 × 6?

Step 1: AI generates three answers

Answer A: 30 Reward: +1

Answer B: 35 Reward: −1

Answer C: 30 Reward: +1

Step 2: GRPO compares them

It sees that A and C received better rewards than B.

Step 3: GRPO updates the model

- It adjusts the model to make good answers more likely.
- It adjusts the model to make poor answers less likely.

Step 4: The AI answers future questions

Next time, the AI is encouraged to give the correct answer, 30, more often.

GSPO -- Group Sequence Policy Optimization

GSPO evaluates and updates the model using the reward for the whole generated answer (the sequence).

Example: 'Explain why 5 × 6 = 30.'

The AI generates a complete answer:

'5 × 6 means adding 5 six times. Therefore, 5 + 5 + 5 + 5 + 5 + 5 = 30.'

The reward system evaluates the whole answer and then update the model (It does not focus on the individual words).

Example: Imagine a teacher checking a complete answer of 5 marks.

1. Student writes the answer.
2. Teacher checks the answer and gives the score.
3. The student learns form feedback.

SAPO -- Soft Adaptive Policy Optimization

SAPO adjusts learning smoothly instead of using a strict cutoff.

Example: You ask AI

'What is 5 × 6?'

The AI generates: 'The answer is 30.'

It generates probability ratio of each token.

Token	**Old probability**	**New probability**	**Ratio**
The	50%	55%	1.1
answer	40%	44%	1.1
30	10%	50%	5.0

Here system considers the individual tokens.

Example:

The → small change → useful → less adjustment.

Answer → small change → useful → less adjustment.

30 → large change → reduce its update.

Example: Imagine teacher giving feedback slowly, like improve the handwriting.

1. Student write the answer.
2. Teacher considers parts of work.
3. Teacher gives appropriate answer.